



DATA 641:
NATURAL LANGUAGE PROCESSING
Dr. Shabnam Tafreshi

FINAL PROJECT - SPRING 2023
May 15, 2023

WASSA 2023 Shared Task
Track 1: Empathy and Emotion Prediction in
Conversations (CONV)

Bishal Regmi

SCIENCE ACADEMY
UNIVERSITY OF MARYLAND, COLLEGE PARK

Contents

1. Introduction	3
2. Dataset	3
2.1. Data Preprocessing	4
3. Multimodal Transformer	4
3.1. Combining Methods	4
4. Naive Implementation	5
4.1. Results and Baseline	5
4.2. Analysis	6
5. Final Implementation	7
5.1. Results and Baseline	8
5.2. Analysis	8
6. Conclusion	9
7. Limitation	9
A. Appendix	11
A.1. Colab Notebooks for the Project	11

1. Introduction

Generally, with Natural Language Processing problems, the primary focus is on the text features. The categorical or numerical features do not usually get much attention, but they might provide additional context or key information to help the model perform better. In this project, the goal is to determine the benefits and draws-backs of using features other than the text features in the context of the Computational Approaches to Subjectivity, Sentiment, and Social Media Analysis (WASSA) 2023 Shared Task [1].

As the name of the shared task suggests, the WASSA 2023 shared task focuses on Empathy Detection, Emotion Classification, and Personality Detection in Interactions [1]. The shared task contains five different tracks, but the track this project focuses on is to predict empathy and emotion at the speech turn in a conversation after having written a reaction essay after reading news articles that contain harm to a person, group, or other [1]. With the text data from the conversation, other feature columns contain information about the speaker, the article, and much more. With the additional information, the goal is to compare just using the text data against using other valid features. This requires the use of a multimodal transformer model, so the Multimodal Toolkit is used [2].

2. Dataset

The data for this project is provided along with the shared task description. The data is collected as shown in Figure 1 [1].

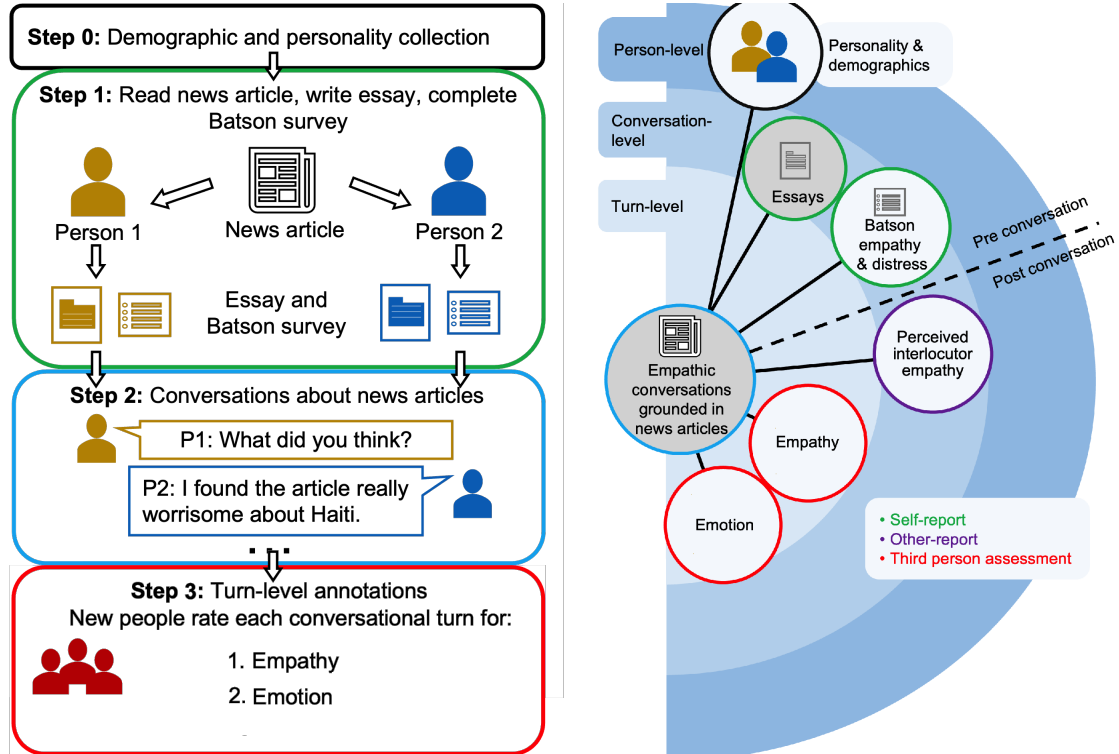


Figure 1: A diagram of how the data collection process works [1]

So, as shown in the figure, this project uses the data produced from the last step in the data collection process. Track one, or the empathy and emotion prediction in conversations track consists of data from conversations between two users that read the same article, and each of their speech turns is annotated in perceived empathy, emotion polarity, and emotion intensity [1].

On average, there are 23 speech turns in each conversation [1]. Along with the conversation data, there are other features in the dataset. There are a total of ten columns with seven independent variables and three target variables. The independent variables are as follows: conversation id, turn id, text, speaker number, article id, speaker id, and essay id. With that, the target variables are emotional polarity, emotion intensity, and empathy. The training set contains 8776 rows, and the development set contains 2400 rows.

2.1. Data Preprocessing

The provided data is mostly clean and does not require much massaging other than converting id columns from floats to integers. The Multimodal Toolkit used in the project also contains a function that helps take the raw data and convert it into the torch dataset that the transformer model can use. It does require that the data be set up in a data frame or files with only the columns that are to be used. In this project, the data with only the desired columns are stored in files for training and evaluation. After that, the data is loaded using the "load_data_from_folder" function from the toolkit [2].

The "load_data_from_folder" function requires that the columns and their data types be specified into the following three categories: text, categorical, and numeric. The text data is processed using "AutoTokenizer" from the Hugging Face library [3]. The auto tokenizer function uses a pre-trained model to tokenize the data, and in this case, the RoBERTa base model is used [4]. The categorical data is processed using either one hot encoding or binary encoding method. The default, binary, is used in this project. Finally, the numeric data is processed using three different methods: "yeo_johnson," "box_cox," and "quantile_normal." The data is processed using the power transformer or the quantile transformer from the Sklearn Library [5]. In this project, the default "yeo_johnson" method is used. With that, most of the work is abstracted, and the output of the function is the features as a "TorchTabularTextDataset," which is a wrapper for the torch dataset [6]. The target values are numeric. Again, as the model can only deal with one target variable at a time, only one target variable is passed at a time. That requires the creation of three models to account for each of the different targets.

3. Multimodal Transformer

Besides the data processing functionality, the multimodal toolkit provides furthermore functionality to combine the data from different data types to create the final multimodal dataset as well as the multimodal transformer model [2]. Figure 2 shows the overall diagram of what the library provides.

The parts about input and data processing are already discussed in Section 2.1. The rest of the diagram shows the additional functionality provided by the library. For the text data, it uses a Hugging Face transformer model as the base model [2]. After that, the output from the Hugging Face transformer and the rest of the processed numerical and categorical data go into the combining module, where there are different ways to combine the data and produce multimodal features for the, in this case, regression layer following the combining layer [2]. So, give a pre-trained model, like RoBERTa, the models get trained using the output of the combining module and transformer [2].

3.1. Combining Methods

There are a variety of ways to combine the data. Without going into too much detail, there are a total of eight different methods. They do exactly as their name suggests. The list of methods includes text only, concatenate, multilayer perceptron on categorical then concatenate, individual multilayer perceptrons on categorical and numerical features and then concatenate, multilayer perceptrons on concatenated categorical and numerical features then concatenate,

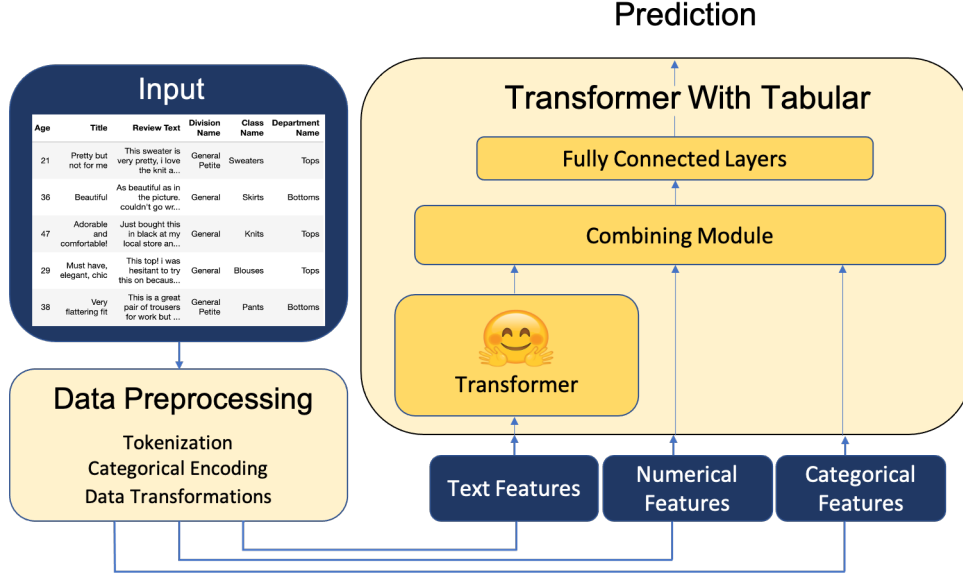


Figure 2: A diagram of how the Multimodal Transformer works [2]

attention on categorical and numerical features, gating on categorical and numerical features then sum, and finally weighted feature sum on transformer categorical and numerical features [2]. All of these different methods are not implemented in the experiment as some do not work with just categorical data in the case of most of the features being IDs rather than true numeric features. Additionally, if all eight methods were to be implemented, that would mean a total of 24 different models that need to be implemented and tested, which would require far more time and computational power than this project can afford. With that, the initial implementation starts with all the different methods other than individual multilayer perceptrons on categorical and numerical features and then concatenate and multilayer perceptrons on concatenated categorical and numerical features then concatenate.

4. Naive Implementation

The initial implementation of this project consists of looking at all the methods other than the two stated in Section 3.1 and creating models for empathy and then taking the method that performs the best for empathy and using that for the other two target variables emotional polarity and emotional intensity.

The baseline is created using the text-only method, which only uses the text feature to create the model. This allows for comparing just using the text versus including the other features. In this case, the method is considered naive as all the other features are used instead of filtering out potentially unnecessary features. The danger with this method is that some of the columns can be highly correlated and could lead to issues. In order to evaluate this model, four different evaluation metrics are used: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Pearson correlation score. With that, the creation, training, and evaluation of the model begins.

4.1. Results and Baseline

The results of the models targeting empathy are in Table 1. The results here are not stable. The local hardware that the models are trained on has unstable Cuda, and therefore most of the results are from only running three to five epochs.

Table 1: Results for Empathy

Method	MSE	RMSE	MAE	Pearson Corr
Baseline (TextOnly)	0.4886	0.6990	0.5471	0.6858
Concat	0.5063	0.7115	0.5557	0.6870
MLP on Cat	0.4952	0.7037	0.5480	0.6846
Attention on Cat and Num*	6.0156	2.4527	2.2614	-0.0254
Gating then Sum**	0.7040	0.8390	0.6907	0.5123
Weighted*	0.6610	0.8130	0.6671	0.5438

Looking at the results, there are a few details that need to be clarified before comparing the results. The default learning rate for the first three models is $1e^{-6}$, but the method names with an asterisk use an initial learning rate of $1e^{-7}$, and the double asterisks denote the use of $1e^{-8}$ as the initial learning rate. This makes it harder to compare all the results with the baseline—especially the last three methods. In order to allow the last three methods to be compared to methods using different learning rates, the baseline is also trained with the different learning rates. The baseline outperformed the last three methods even with the lower learning rates trained for similar five epochs. So, due to the complexity that they add along with their poor performance, the last three methods are ignored from further analysis. If the models were trained for more epochs, the results should be different. Therefore, in future research, it might be fruitful also to include those methods again. But for now, the focus is on only the first three methods, which include the baseline. The interesting result here is that looking at the Pearson correlation score, the concat method does slightly better than the baseline, but it is also visible that the other error values are also higher. So, along with the Pearson correlation score, the errors also increase. This is to be discussed later. After noting that the last three methods be ignored and with the concatenate method doing the best compared to the other methods, only the concatenate combining method is to be used for the other two target variables.

Table 2: Results for Emotional Intensity and Emotional Polarity

Target:Method	MSE	RMSE	MAE	Pearson Corr
Emotional Intensity: Baseline (TextOnly)	0.3198	0.5654	0.4416	0.7556
Emotional Intensity: Concat	0.3265	0.5714	0.4468	0.7572
Emotional Polarity: Baseline (TextOnly)	0.1658	0.4072	0.3012	0.7640
Emotional Polarity: Concat	0.1682	0.4102	0.3012	0.7677

The results of the emotional intensity and polarity models are in Table 2. Again only the baseline and the concatenate methods are used, as the others are filtered out after looking at the results from the empathy models. The results are yet again similar. The concatenation method does, in fact, slightly improve the model’s performance based on the Pearson correlation score, but the errors also increase as well. This further highlights the weakness of the naive implementation.

4.2. Analysis

To further analyze and take a deeper look at the results, it is clear that the results are conflicting. While the score used by the shared task to evaluate the model does improve, the errors themselves do not improve. This could be due to various factors ignored in the naive implementation. With that, the current results are not reliable. The issue with the errors being higher, along with the correlation, could be due to multiple factors. One is the potential outliers in the data causing both error and correlation to increase. The Pearson correlations core is heavily affected by outliers. With the addition of the new feature variables, there could be new outliers introduced in the results that could both increase the error while also increasing the Pearson correlation score. It

could also be a nonlinear relationship being misjudged by the correlation coefficient. It could also be that there exists multicollinearity between the independent variables, and this can also lead to the current scenario.

Additionally, it could also be related to how the metrics vary. Looking at the different metrics, the error metrics, such as MSE, RMSE, MAE, etc., measure the average magnitude of the errors in the set of predictions without considering their direction. This provides a general idea of how far off the predictions are on average. In contrast, the Pearson correlation score measures the linear relationship between the predicted and true values. A score of 1 implies a perfect positive linear relationship, a score of -1 implies a perfect negative linear relationship and a score of 0 implies no linear relationship. Adding new features to your model can potentially improve the linear relationship between the predictions and the true values, which increases the Pearson correlation score. However, introducing the new features also introduces additional complexity, causing the predictions to be further off than before. So, after adding new features, the predictions might have spread out, with some predictions being much further from the true values, which increases the error, but others being much closer, which increases the correlation. So, while the model might make larger mistakes on some samples leading to greater error, it might also capture the overall trend in the data more accurately, resulting in a higher correlation.

With that, error metrics and correlation coefficients provide different insights into your model's performance, and it's not uncommon for them to move in opposite directions as the model is modified. The main information to extract from this naive implementation is that further analysis is required.

5. Final Implementation

Understanding that using all the features along with the text features might not be the best idea, further analysis is done. It is discovered that some of the feature columns are highly correlated, like columns speaker id and essay id, which are different values but one-to-one correlated. Other columns are not exactly one-to-one correlated but have a high correlation. With that, only the article id column is used. The experiment now focuses on only using the concatenation method along with the baseline as it is slightly better than the baseline in the naive implementation, while other methods outright failed. This also allows the article id to be considered both a categorical and numerical column and tested to see if there is a difference in the result. Additionally, while analyzing, it is also discovered that there are unseen article ids in the evaluation set. This is bound to cause issues when the model is evaluated with the evaluation set. The categorical method using binary encoding should solve some of the problems, such as the differing number of columns added when using one-hot encoding. However, it also means that the binary encoding for the evaluation set does not match directly with what is encoded in the training set. This issue is not resolved but left for further analysis in the future. Additionally, using the article id as numeric should also give an interesting interpretation of the dataset. With that, it is clear that adding the other features along with the text features drastically increases the complexity of the model, as various factors must be considered.

With that, a total of nine different models are created. For each of the three target variables, a baseline, a model using the concatenation method where article id is considered a categorical, and then finally, a model using the concatenation method where article id is considered a numerical. Similar to before, in order to evaluate the models, four different evaluation metrics are used: MSE, RMSE, MAE, and Pearson correlation score. Additionally, due to the instability caused by local hardware, the project is moved to the Colab workspace for the rest of the project.

5.1. Results and Baseline

The results from the nine different models are in Table 3. In Colab, the model is far more stable than the local hardware runs. Each model is trained up to 30 epochs, and the best results with respect to the Pearson correlation score are in Table 3. There were a total of 54 different plots generated from the results. Instead of including all the plots in the report, they are stored in the Colab notebook. The link to the notebook is in Appendix A.1. To summarize the main plots, which consist of plots of loss in training and evaluation, most of the plots show that after a few epochs, the evaluation loss flattens out while the training loss keeps decreasing. At most, it takes less than 15 epochs for the best results to be achieved. The plots also contain a point at their max and min for better visualization. Additionally, in some cases, the best Pearson correlation score does not correspond to the lowest errors, while in some, they match up exactly. So, as discussed earlier in Section 4.2, this pattern is not just related to adding new features or issues with the data, but it is just how the different metrics work. The correlation score can work in the opposite with error metrics.

Table 3: Evaluation Results

epoch	MSE	RMSE	MAE	Pearson Corr	Method	Target Variable
9	0.4843	0.6959	0.5446	0.6917	TextOnly	Empathy
11	0.4995	0.7068	0.5485	0.6837	Concat_Cat	Empathy
11	0.4861	0.6972	0.5432	0.6926	Concat_Num	Empathy
8	0.1578	0.3972	0.2904	0.7757	TextOnly	Emotional_Polarity
7	0.1543	0.3928	0.2866	0.7859	Concat_Cat	Emotional_Polarity
14	0.1633	0.4041	0.2936	0.7762	Concat_Num	Emotional_Polarity
9	0.3179	0.5639	0.4366	0.7620	TextOnly	Emotional_Intensity
15	0.3269	0.5717	0.4417	0.7557	Concat_Cat	Emotional_Intensity
9	0.3234	0.5686	0.4429	0.7594	Concat_Num	Emotional_Intensity

The results are interesting yet again. There is no clear answer to which method is the best. In terms of Pearson correlation score, using the concatenation method with article id as a numeric is the best for empathy, using the concatenation method with article id as a categorical is the best for emotional polarity, and finally, it is actually best to just use text only for emotional intensity.

5.2. Analysis

So, for the three different targets, three different methods are the best. Additionally, looking at the errors, with the analysis from Section 4.2, it is clear that it is normal for the Pearson correlation score to go up with the error, so the results are reliable. Additionally, compared to the results from Table 1 or Table 2 where the concatenation method always had a higher correlation score, those scores are only with five epochs at most, whereas in almost all the cases with the higher correlation score they are above five epochs with the min being seven epochs in Table 3. With that, all the results are reliable. It only leaves the project with further questions. So with each target variable, the results can vary, and additionally, this opens the door to using the other different combination methods to determine if they might perform better, as it is clear that the methods do not perform consistently and vary with each scenario. Using all the different non-text variables might not be a good idea, but using the article id with the different combination methods might further bring promising results.

6. Conclusion

To sum it up, blindly adding non-text features along with the text feature, as done in the naive implementation, does not improve the model, and it actually introduces instability and leads to unreliable results. As visible in the results from the naive implementation, it might seem like the model with the concatenation method always performs better; the instability in the model takes away from the results. However, even with careful filtering and picking the right features to include and properly encoding, it likewise might not help the model perform better. In some situations, it actually adds complexity, as above in the case of unseen data when trying to use the article id as a categorical variable; The mapping is bound to be error-prone when using binary. However, encoding with one hot encoding is also bound to cause issues with the input size of the evaluation set and the training set not having the same number of columns. So, this just adds additional complexity while it might not really help the model perform better, as visible in the case of emotional intensity, where the baseline outperforms the other two models. With that, when creating a Natural Language Processing model, the decision to use numerical and categorical features along with text features depends largely on the specific problem at hand. The numerical or categorical features can often provide valuable context that is not available in the text data alone, like the article that the conversation is talking about in this case. However, would using the topic or a one-sentence summary from the essay rather than the id help the model perform better? So, it would just be text data rather than numeric or categorical. Additionally, it is worth noting that incorporating these features, as seen above, can also introduce additional complexity into the model, and it might not always be necessary. It's also possible that the model could learn to rely too heavily on these features, which could potentially hurt its ability to generalize to new data. In conclusion, there is no one-size-fits-all answer to using non-text features with text data. It is often best to try several different approaches and see what works best for the specific use case as done with the final implementation. This method can be further improved in the future with additional combination methods and better solutions to tackle the unseen data issue.

7. Limitation

This project faced a fair amount of limitations. The primary limitation is the time limit. Due to other factors outside of the project, only a minimal amount of time could be dedicated to this project. With the limited time came the pressure of using hardware that is capable of handling heavy computational load and being able to process it all rapidly. Sadly, this led to the project initially being implemented on the more powerful local hardware instead of Colab, but the local hardware was extremely unstable. But the instability was only discovered late into the project. The instability in the hardware leads to a loss in much of the limited time trying to fine tune the model but failing due to the issue being the hardware rather than the model. So most of the issue with gradient explosion or vanishing was due to hardware and unstable Cuda installation. The issues popped up as "address misalignment" issues and additional run-time errors that are hard to debug. Additionally, there were issues in the Multimodal Toolkit where it has some specific requirements that are not obvious and requires digging through the library code. Overall, the hardware and computational powers, along with the time limit, were a big factor in preventing this project from excelling further.

On top of that, the lack of common sense in terms of NLP knowledge and general best practice when working with transformer models also added to the issues by taking away time trying to learn the basics of first-time implementation rather than focusing on the problem at hand. Especially in the case of the first implementation, the lack of common sense there greatly harmed the overall outlook of the project. Thankfully, the final implementation, with the knowledge gained from mistakes in the first implementation, helped better analyze and improve the project.

References

- [1] D. Omitaomu, S. Tafreshi, T. Liu, S. Buechel, C. Callison-Burch, J. Eichstaedt, L. Ungar, and J. Sedoc, “Empathic conversations: A multi-level dataset of contextualized conversations,” 2022.
- [2] K. Gu and A. Budhkar, “A package for learning on tabular and text data with transformers,” in *Proceedings of the Third Workshop on Multimodal Artificial Intelligence*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2021, pp. 69–73. [Online]. Available: <https://www.aclweb.org/anthology/2021.maiworkshop-1.10>
- [3] “Hugging Face Transformers Documentation,” <https://huggingface.co/docs/transformers/index>.
- [4] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” 2019.
- [5] “Scikit-Learn Preprocessing Documentation: Encoding Categorical Features,” <https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features>.
- [6] “PyTorch Tutorials: Data Loading and Processing Tutorial,” https://pytorch.org/tutorials/beginner/basics/data_tutorial.html.

A. Appendix

A.1. Colab Notebooks for the Project

The data processing and model training/evaluation is done in the following notebook: <https://drive.google.com/file/d/1y4afggFTNJyW6z0yPoUWVDU040H7mjXb/view?usp=sharing>

The result visualization and analysis are done in the following notebook: <https://colab.research.google.com/drive/1ujtF0sqZzEZ18aj42s07g62K2vD02DT9?usp=sharing>

Both of the notebooks require access to your drive with specific files included. These links do not provide that. All of the needed files will be submitted with the paper. The project requires adjustments and file path changes before the reader can run it.